实验七：运行时环境与内存映像设计

计算机2203 苏佳琪 2226124199

代码仓库：https://github.com/Su-Jiaqi/Compiler-Project

一、实验目的

本次实验在前面词法分析、语法分析、语义分析以及中间代码生成实验的基础上，进一步理解编译器后端中的运行时环境设计。实验的主要目标是根据语义分析阶段得到的符号表信息，为程序中的变量、临时变量和函数等对象设计合理的存储格式，并构建对应的内存映像，为后续目标代码生成或汇编代码生成提供基础。

通过本次实验，可以进一步理解编译器从源程序到目标程序的完整流程。前面的实验主要关注词法、语法和语义层面的分析，而实验七开始关注程序运行时的数据存储方式，例如变量在内存中的地址、偏移量、作用域以及不同类型变量所占用的空间等内容。

二、实验内容

本次实验主要完成以下内容：

1. 读取前面实验生成的符号表信息；
2. 根据变量类型和作用域，为变量分配内存地址或偏移量；
3. 设计简单的运行时存储格式；
4. 构建内存映像，输出变量名、类型、作用域、地址偏移等信息；
5. 为实验八的目标代码生成提供基础。

由于本课程实验七和实验八在教学安排中通常合并进行，且验收程序允许实现到中间代码生成阶段，因此本项目在实验六已经完成了四元式/三地址码形式的中间代码输出。本实验在此基础上重点说明如何利用已有的符号表和中间代码进一步扩展运行时内存映像。

三、实验环境

本实验使用统一的编译器工程项目完成，环境如下：

- 操作系统：Windows
- 编译环境：MSYS2 UCRT64 / g++
- 编程语言：C++
- 工程管理：Makefile
- 编辑器：Cursor / VS Code
- 输入文件：`input/test.src`
- 输出目录：`output/`

项目采用统一工程组织方式，前面实验的词法分析、语法分析、语义分析和中间代码生成模块都在同一个项目中持续扩展，便于后续实验复用已有结果。

四、相关文件说明

本实验相关的主要文件如下：

```
compiler_project/
├── include/
|   ├── lexer.h
|   ├── ast.h
|   ├── semantic.h
|   └── ir.h
├── src/
|   ├── lexer.cpp
|   ├── ast.cpp
|   ├── semantic.cpp
|   ├── ir.cpp
|   └── main.cpp
├── input/
|   └── test.src
├── output/
|   ├── ast.txt
|   ├── symbols.txt
|   └── ir.txt
└── Makefile
```

其中：

- `input/test.src` 是输入源程序；
- `output/ast.txt` 是语法树输出结果；
- `output/symbols.txt` 是符号表输出结果；
- `output/ir.txt` 是中间代码输出结果；
- 符号表中的变量信息可作为实验七内存映像构建的基础；
- 四元式中间代码可作为实验八目标代码生成的基础。

五、实验原理

运行时环境是程序执行时所依赖的内存组织方式。编译器在完成语义分析后，需要知道程序中有哪些变量、变量类型是什么、变量位于哪个作用域中，以及它们在运行时如何存储。

对于一个简单的类 C 语言子集，可以采用较为简单的内存模型：

```
全局区 / 符号表区
局部变量区
临时变量区
中间代码区
```

在本实验中，变量主要来自语义分析阶段生成的符号表。每个符号表项可以包含以下信息：

```
变量名
变量类型
变量作用域
变量地址或偏移量
```

例如，对于如下源程序：

```
int a;
int b;
a = 3;
b = a + 5;
print b;
```

可以构建类似如下的符号表和内存映像：

```
Name    Type    Scope    Offset
a       int     global   0
b       int     global   4
```

其中 `a` 和 `b` 都是整型变量，每个变量占用 4 字节，因此变量 `a` 的偏移量为 0，变量 `b` 的偏移量为 4。

如果后续生成目标代码，就可以根据这些偏移量访问变量。例如中间代码：

```
t1 = a + 5
b = t1
```

可以进一步翻译为某种汇编风格的目标代码。

六、实验步骤

1. 编译项目

首先进入项目目录：

```
cd E:\course\4.2\CompilerProj\compiler_project
```

然后执行编译命令：

```
make clean
make
```

如果编译成功，会生成可执行文件：

```
compiler.exe
```

2. 运行语义分析和中间代码生成

由于本项目的符号表、AST 和中间代码统一由 `ir` 模式生成，因此运行如下命令：

```
.\compiler.exe ir input\test.src output
```

该命令会读取 `input/test.src`，完成词法分析、简单语法结构构建、语义分析、符号表生成和中间代码生成。

3. 查看符号表结果

运行命令：

```
Get-Content output\symbols.txt
```

该文件中保存了语义分析阶段得到的符号表信息。符号表是实验七构建运行时内存映像的基础。

4. 查看中间代码结果

运行命令：

```
Get-Content output\ir.txt
```

该文件中保存了实验六生成的四元式或三地址码形式的中间代码。中间代码可以在实验八中进一步翻译为目标代码或汇编代码。

七、实验结果

本实验运行命令如下：

```
.\compiler.exe ir input\test.src output
```

输出文件包括：

```
output/ast.txt
output/symbols.txt
output/ir.txt
```

其中，`symbols.txt` 可以作为运行时内存映像设计的依据，`ir.txt` 可以作为后续目标代码生成的输入。

实验运行后，查看符号表：

```
Get-Content output\symbols.txt
```

可以得到程序中的变量信息，包括变量名、类型和作用域等内容。根据这些信息，可以为变量分配对应的内存位置。

查看中间代码：

```
Get-Content output\ir.txt
```

可以得到程序对应的四元式或三地址码。中间代码中的变量和临时变量均可以根据符号表和运行时存储规则映射到具体内存位置。

```
PS E:\course\4.2\CompilerProj\compiler_project> .\compiler.exe ir input\test.src output
semantic/IR done
PS E:\course\4.2\CompilerProj\compiler_project> Get-Content output\symbols.txt
name    type    offset
a       int     0
b       int     4
c       float   8
PS E:\course\4.2\CompilerProj\compiler_project> Get-Content output\ir.txt
0: (=, 3, , a)
1: (=, 5, , b)
2: (=, 2.5, , c)
3: (*, b, 2, t1)
4: (+, a, t1, t2)
5: (=, t2, , a)
6: (<, a, 20, t3)
7: (jz, t3, , L1)
8: (print, a, , )
9: (j, , , L2)
10: (label, , , L1)
11: (print, b, , )
12: (label, , , L2)
13: (label, , , L3)
14: (<, a, 25, t4)
15: (jz, t4, , L4)
16: (+, a, 1, t5)
17: (=, t5, , a)
18: (j, , , L3)
19: (label, , , L4)
20: (return, a, , )
PS E:\course\4.2\CompilerProj\compiler_project>
PS E:\course\4.2\CompilerProj\compiler_project>
```

八、运行时内存映像设计

本项目可以采用如下简单的内存映像设计方式：

```
Offset    Name    Type    Scope
0         a       int     global
4         b       int     global
8         t1      int     temp
12        t2      int     temp
```

其中：

- 普通整型变量占用 4 字节；
- 浮点型变量可以设置为占用 4 字节或 8 字节；
- 临时变量由中间代码生成阶段产生；
- 每个变量按照声明顺序依次分配偏移量；
- 后续目标代码生成时，可以通过偏移量访问变量。

本实验中的内存映像设计仍然是教学版和简化版，主要用于展示编译器后端从符号表到运行时存储分配的基本过程。

九、实验分析

本次实验与前面实验之间具有明显的连续关系。实验二生成 Token 流，实验三和实验四完成语法分析相关内容，实验五生成 AST 和符号表，实验六生成中间代码，而实验七则进一步利用符号表和中间代码进行运行时环境设计。

其中，符号表在实验七中具有核心作用。符号表不仅保存变量名和类型，还可以扩展保存变量的作用域、地址偏移量、是否为临时变量等信息。运行时内存映像本质上就是对符号表中各类对象进行存储空间分配的结果。

由于本项目目前主要实现到中间代码生成阶段，因此实验七的重点是说明如何基于已有符号表进一步扩展内存映像。后续如果继续完善，可以在符号表项中增加 `offset` 字段，并在语义分析或中间代码生成结束后统一分配变量地址，最终输出完整的 `memory_map.txt` 文件。

十、实验总结

通过本次实验，我进一步理解了编译器后端中运行时环境和内存映像的作用。前面的词法分析、语法分析和语义分析主要解决的是源程序是否符合语言规则的问题，而运行时环境设计则进一步关注程序在执行时如何组织和访问数据。

本实验基于前面生成的符号表和中间代码，设计了简单的变量存储方式和地址偏移分配思路。虽然当前实现仍然是简化版本，但已经能够体现从符号表到内存映像的基本流程，也为后续实验八中的目标代码生成提供了基础。

本次实验说明，一个完整编译器不仅需要完成前端分析，还需要在后端阶段将变量、临时变量和中间代码映射到具体的运行时存储结构中。通过本次实验，我对编译器整体流程有了更加完整的认识。